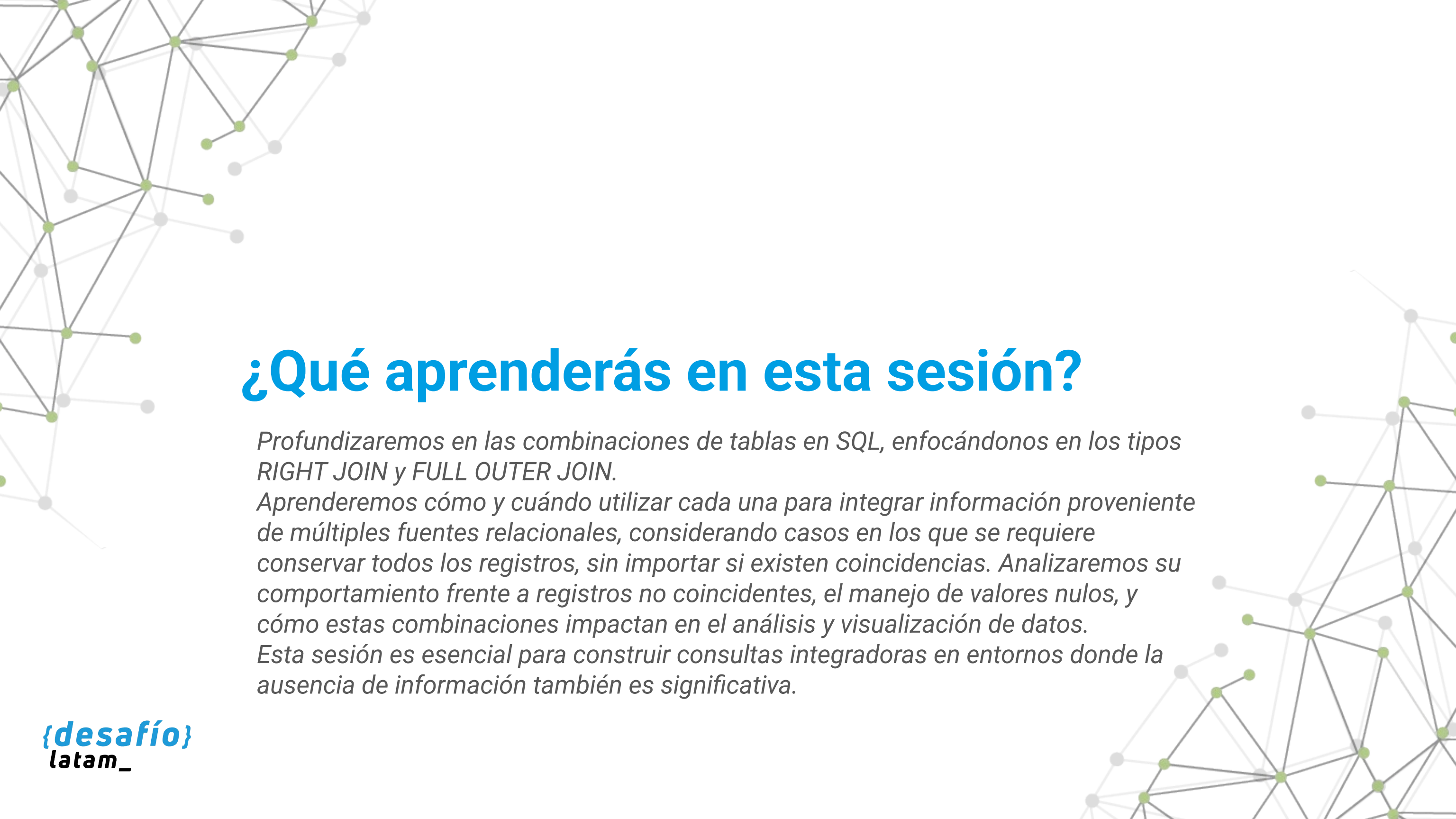




Consultas Avanzadas y Relaciones

**Clase 3 - Consultas de selección con tablas relacionadas
(Parte II)**



¿Qué aprenderás en esta sesión?

Profundizaremos en las combinaciones de tablas en SQL, enfocándonos en los tipos `RIGHT JOIN` y `FULL OUTER JOIN`.

Aprenderemos cómo y cuándo utilizar cada una para integrar información proveniente de múltiples fuentes relacionales, considerando casos en los que se requiere conservar todos los registros, sin importar si existen coincidencias. Analizaremos su comportamiento frente a registros no coincidentes, el manejo de valores nulos, y cómo estas combinaciones impactan en el análisis y visualización de datos.

Esta sesión es esencial para construir consultas integradoras en entornos donde la ausencia de información también es significativa.

Desafío inicial

Eres parte del equipo de análisis en una consultora que trabaja con datos de estudiantes y sus rendimientos académicos. Los datos de los estudiantes se encuentran en una tabla y las evaluaciones aplicadas en otra. La gerencia desea un informe con todos los estudiantes, incluso aquellos que aún no han sido evaluados, así como todas las evaluaciones, aunque no estén asignadas a ningún estudiante (por errores en la carga, por ejemplo).

¿Cómo construir una consulta que incluya todos los registros de ambas tablas, sin perder información? ¿Qué tipo de combinación es más apropiada para este caso? ¿Qué implicaciones tiene en términos de análisis la presencia de NULL en los resultados?

Para resolverlo, necesitas dominar las combinaciones RIGHT JOIN y FULL OUTER JOIN y comprender cómo se comportan ante relaciones incompletas.

Combinaciones externas adicionales en SQL: RIGHT JOIN y FULL OUTER JOIN

¿Por qué necesitamos combinaciones externas más allá del LEFT JOIN?

Las combinaciones externas permiten incluir registros que no tienen correspondencia en la tabla relacionada. Mientras LEFT JOIN conserva todos los registros de la tabla izquierda, RIGHT JOIN hace lo propio con la tabla derecha. FULL OUTER JOIN incluye todos los registros de ambas tablas, combinando cuando hay coincidencias y dejando valores nulos cuando no las hay.



Diagnosticar cobertura de información entre dos tablas.



Auditar relaciones incompletas.

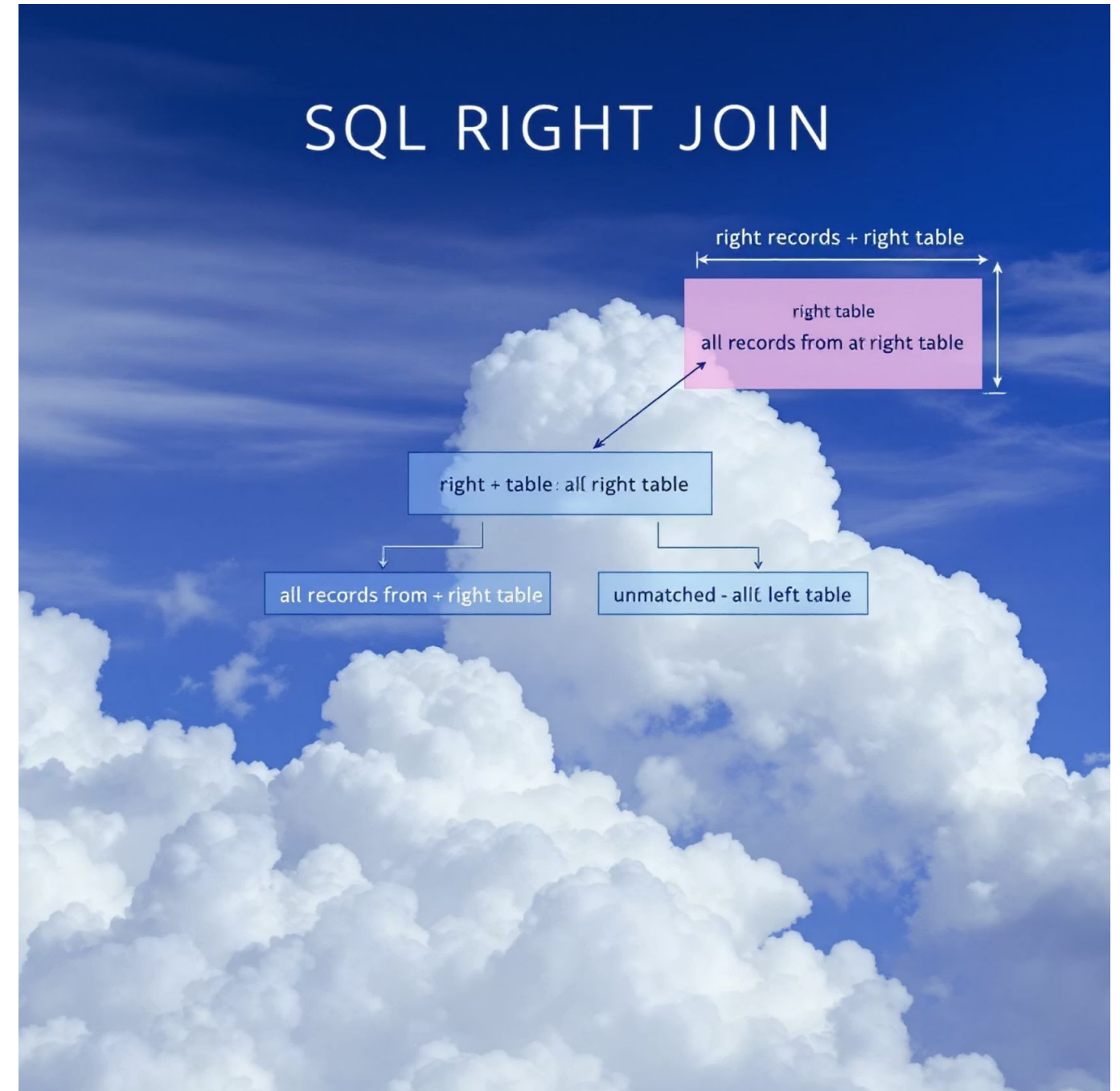


Construir reportes que muestren ausencias de datos (por ejemplo, estudiantes sin evaluaciones o evaluaciones sin asignación).

RIGHT JOIN: manteniendo todos los registros de la tabla derecha

Un RIGHT JOIN devuelve todas las filas de la tabla de la derecha (tabla2), combinándolas con las filas correspondientes de la izquierda (tabla1). Si no hay coincidencia, los valores de la tabla izquierda se llenan con NULL.

```
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
RIGHT JOIN evaluaciones e ON s.id_estudiante =
e.id_estudiante;
```



Contexto productivo

Útil en contextos donde se debe auditar la carga de evaluaciones

Útil en contextos donde se debe auditar la carga de evaluaciones y asegurar que ninguna quede huérfana, aunque aún no esté asociada a un estudiante.

Errores comunes

Interpretar los NULL como errores, cuando en realidad reflejan una relación no existente.

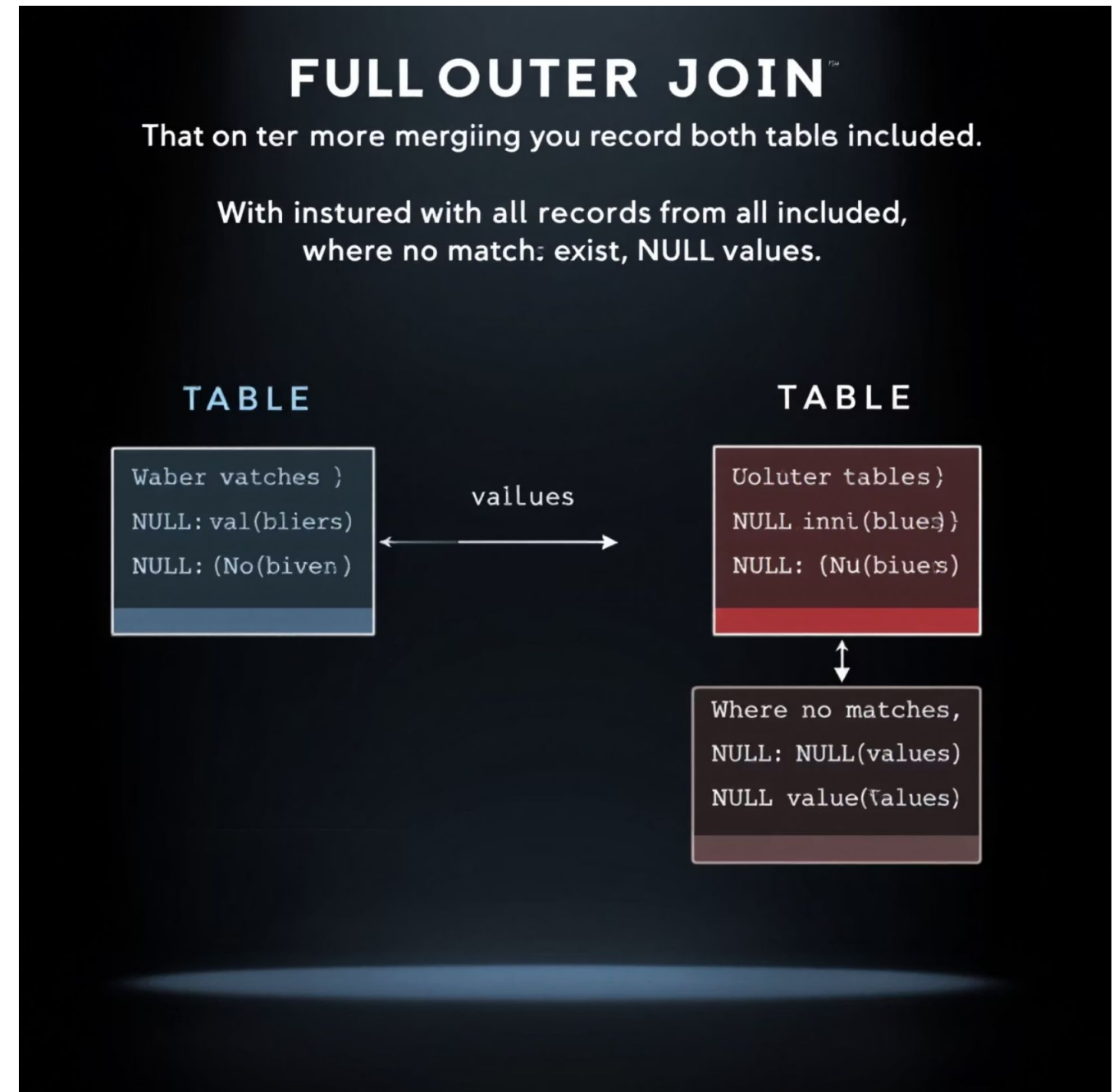
Olvidar revisar la dirección de la combinación: si lo que importa es mantener los registros de evaluaciones, la tabla evaluaciones debe ir a la derecha.

FULL OUTER JOIN: integración total sin pérdidas

Un FULL OUTER JOIN conserva todos los registros de ambas tablas.

Cuando hay coincidencia entre claves, las filas se combinan. Cuando no, los valores de la tabla faltante se llenan con NULL.

```
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
FULL OUTER JOIN evaluaciones e ON
s.id_estudiante = e.id_estudiante;
```





Aplicación práctica

Permite construir un reporte completo, detectando tanto estudiantes sin evaluación como evaluaciones sin asignación. Ideal para diagnósticos y conciliaciones de datos entre sistemas.

Limitación técnica

No todos los motores de base de datos soportan FULL OUTER JOIN directamente (por ejemplo, MySQL). En esos casos puede emularse con UNION de un LEFT JOIN y un RIGHT JOIN excluyendo las coincidencias duplicadas.

```
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
LEFT JOIN evaluaciones e ON s.id_estudiante = e.id_estudiante
UNION
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
RIGHT JOIN evaluaciones e ON s.id_estudiante = e.id_estudiante;
```

Comparación visual de combinaciones JOIN

Tipo de JOIN	Registros mantenidos	Coincidencias requeridas	Uso recomendado
INNER JOIN	Solo coincidencias	Sí	Reportes donde solo interesa la intersección de datos.
LEFT JOIN	Todos de la izquierda	No	Cuando se desea mantener el total de registros base.
RIGHT JOIN	Todos de la derecha	No	Para validar registros complementarios (auditorías, cruces auxiliares).
FULL OUTER JOIN	Todos	No	Para diagnósticos, reportes generales o conciliaciones.

Tabla 1. Tipos de combinaciones JOIN y su comportamiento

Fuente: Desafío Latam

Buenas prácticas

Recomendaciones

-  Clarificar el objetivo del análisis antes de elegir el tipo de combinación.
-  Documentar el comportamiento esperado respecto a los NULL.
-  Validar si el sistema de base de datos soporta FULL OUTER JOIN.

Métricas clave

?%

**Proporción de registros sin
coincidencia**

?

**Volumen de NULL
generados**

Actividad guiada:

Combinando información de estudiantes y evaluaciones



Objetivo de la actividad

Aplicar combinaciones RIGHT JOIN y FULL OUTER JOIN en SQL para integrar registros de dos tablas relacionadas, identificando coincidencias y ausencias de datos, y evaluando el impacto analítico de cada tipo de combinación.



Situación problema

Eres parte del equipo de analistas de datos de una institución educativa. Recientemente se ha detectado que algunas evaluaciones fueron cargadas al sistema sin estar vinculadas a ningún estudiante, mientras que otros estudiantes aún no han rendido evaluaciones.

El equipo directivo necesita un reporte que muestre toda la información disponible, incluyendo los casos donde los datos estén incompletos o las relaciones no existan.

Tu tarea es combinar las tablas estudiantes y evaluaciones, aplicando distintos tipos de combinación para obtener una vista completa de la situación, sin perder ningún registro, y analizar los patrones de ausencia de datos.



Instrucciones

Crear las tablas base:

```
CREATE TABLE estudiantes (  
  id_estudiante INT,  
  nombre VARCHAR(50)  
);  
  
CREATE TABLE evaluaciones (  
  id_evaluacion INT,  
  id_estudiante INT,  
  fecha_evaluacion DATE  
);
```

Insertar datos:

```
INSERT INTO estudiantes VALUES  
  (1, 'Ana'),  
  (2, 'Luis'),  
  (3, 'Marcela'),  
  (4, 'Diego');  
  
INSERT INTO evaluaciones VALUES  
  (101, 1, '2024-06-15'),  
  (102, NULL, '2024-06-20'),  
  (103, 3, '2024-07-01'),  
  (104, 5, '2024-07-02');
```

Aplicar RIGHT JOIN y FULL OUTER JOIN

Aplicar RIGHT JOIN:

```
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
RIGHT JOIN evaluaciones e ON s.id_estudiante =
e.id_estudiante;
```

Aplicar FULL OUTER JOIN:

```
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
FULL OUTER JOIN evaluaciones e ON s.id_estudiante =
e.id_estudiante;
```

Analizar y discutir:

¿Qué diferencias observas entre ambas combinaciones?, ¿Qué implican los valores NULL en cada caso?, ¿Qué registros se perderían si usáramos INNER JOIN? ¿Qué combinación se ajusta mejor al objetivo de informar sin omisiones?

Materiales necesarios



Motor de base de datos compatible

PostgreSQL, SQL Server, u otro con soporte JOIN.



Editor o entorno de ejecución SQL

DBeaver, pgAdmin, SQL Fiddle, etc.



Código base

Proporcionado por el relator.

Actividad práctica autónoma:

**Integración de datos de
estudiantes y evaluaciones**



Objetivo de la actividad

Aplicar combinaciones externas (RIGHT JOIN y FULL OUTER JOIN) para integrar dos tablas relacionales con registros incompletos, analizando cómo se comportan estas combinaciones frente a claves faltantes y su impacto en el análisis de datos educativos.

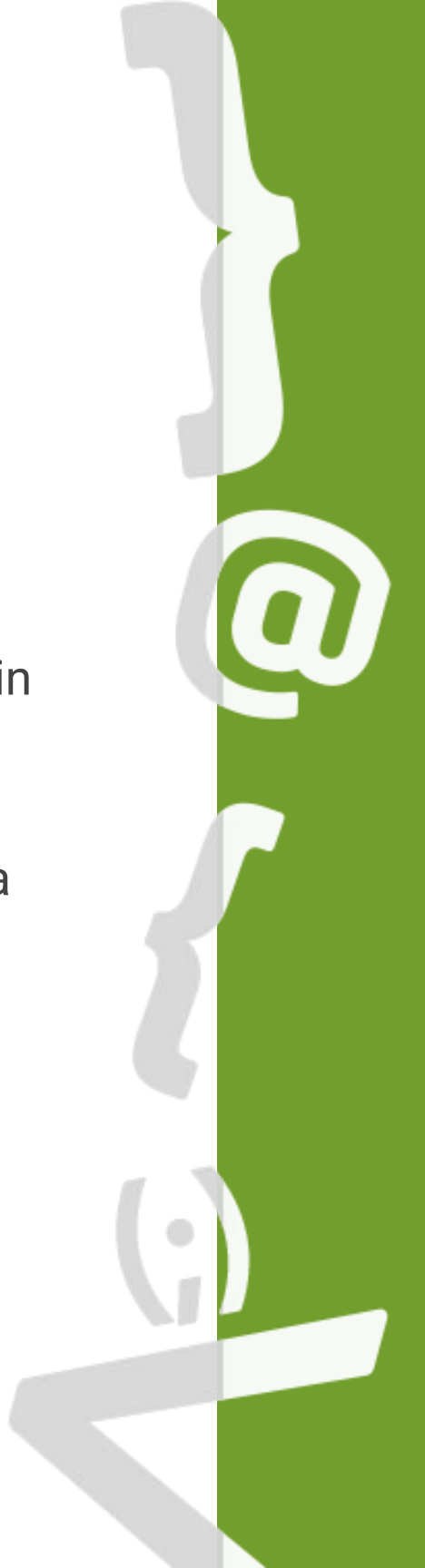


Contexto

Trabajas con una base de datos institucional que contiene información de estudiantes y evaluaciones aplicadas.

Sin embargo, algunos estudiantes aún no rinden evaluaciones, y algunas evaluaciones fueron cargadas sin vinculación directa con un estudiante.

La jefatura te solicita un reporte completo donde se visualicen todos los registros disponibles, sin pérdida de información, para decidir acciones de regularización.



Instrucciones para desarrollar la actividad de forma autónoma

Crea las siguientes tablas e inserta los datos:

```
-- Tabla estudiantes
CREATE TABLE estudiantes (
  id_estudiante INT,
  nombre VARCHAR(50)
);

INSERT INTO estudiantes VALUES
  (1, 'Pedro'),
  (2, 'Laura'),
  (3, 'Ignacio'),
  (4, 'Fernanda');

-- Tabla evaluaciones
CREATE TABLE evaluaciones (
  id_evaluacion INT,
  id_estudiante INT,
  fecha_evaluacion DATE
);

INSERT INTO evaluaciones VALUES
  (101, 1, '2024-06-10'),
  (102, NULL, '2024-06-12'),
  (103, 3, '2024-06-15'),
  (104, 5, '2024-06-20');
```



Consultas y análisis

Ejecuta una consulta con RIGHT JOIN:

```
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
RIGHT JOIN evaluaciones e ON
s.id_estudiante = e.id_estudiante;
```

Ejecuta una consulta con FULL OUTER JOIN:

```
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
LEFT JOIN evaluaciones e ON s.id_estudiante = e.id_estudiante
UNION
SELECT
    s.nombre,
    e.fecha_evaluacion
FROM estudiantes s
RIGHT JOIN evaluaciones e ON s.id_estudiante = e.id_estudiante;
```

Analiza y registra:

Tipo de JOIN	Resultado clave	¿Qué detectas?
RIGHT JOIN	Evaluaciones sin estudiante	
FULL OUTER JOIN	Estudiantes sin evaluación y evaluaciones sin estudiante asignado	



Resumen



Propósito y funcionamiento

Comprendimos el propósito y funcionamiento de las combinaciones externas RIGHT JOIN y FULL OUTER JOIN en SQL.



Auditoría de registros

Aplicamos consultas SQL que permiten identificar y auditar registros huérfanos.



Integración de datos

Analizamos cómo estas combinaciones permiten integrar datos de múltiples tablas conservando registros sin coincidencia.



Emulación de JOIN

Aprendimos a emular un FULL OUTER JOIN en motores que no lo soportan directamente.



Buenas prácticas

Identificamos buenas prácticas para aplicar combinaciones según el contexto productivo.



Próxima sesión...

En la siguiente sesión trabajaremos con consultas agrupadas, explorando cómo resumir y analizar datos mediante funciones de agregación (MIN, MAX, AVG, COUNT) y la sentencia GROUP BY.

*Esto nos permitirá generar reportes estadísticos, extraer tendencias por categorías y responder preguntas como:
¿Cuál fue la calificación promedio por curso? ¿Cuántos estudiantes rindieron evaluación por semana?*

Preguntas de cierre

1

¿Cuál es la diferencia entre un LEFT JOIN y un RIGHT JOIN, y cuándo conviene usar uno u otro?

2

¿Qué desafíos presenta el uso de FULL OUTER JOIN en motores de base de datos que no lo soportan?

3

¿Cómo puedes detectar registros sin correspondencia en una combinación externa?

4

¿Qué implicaciones tiene la presencia de valores nulos en una tabla combinada?

5

¿Por qué es importante validar el comportamiento de las combinaciones antes de integrarlas a procesos de análisis o visualización?

Referencias bibliográficas



Oppel, A. J.

Databases Demystified: A Self-Teaching Guide
(3rd ed.). McGraw-Hill Education. (2020)



Groff, J. R., & Weinberg, P. N.

SQL: The Complete Reference (3rd ed.).
McGraw-Hill Education. (2014)



PostgreSQL Documentation 16

PostgreSQL Global Development Group. (2024).
<https://www.postgresql.org/docs/>



Oracle SQL Language Reference

Oracle. (2024).
<https://docs.oracle.com/en/database/>



Microsoft Transact-SQL Reference

Microsoft. (2024).
<https://learn.microsoft.com/en-us/sql/t-sql/>